

6b. Analyse the impact of Zero-shot, Few-shot, and Chain-of-Thought Prompting on the reasoning capabilities of Large Language Models and identify patterns in hallucinations and logical errors

AIM

To analyze and compare the performance of Zero-shot, Few-shot, and Chain-of-Thought prompting techniques on Large Language Models using the GSM8K dataset, by implementing them in Python and evaluating their accuracy in solving reasoning-based problems.

IMPORT LIBRARIES

```
import re
import pandas as pd
import matplotlib.pyplot as plt
from datasets import load_dataset
from transformers import pipeline
```

LOAD DATASET

```
dataset = load_dataset("openai/gsm8k", "main")
samples = dataset["test"].select(range(5))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:      7.93k/? [00:00<00:00, 577kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
main/train-00000-of-00001.parquet: 100%                2.31M/2.31M [00:00<00:00, 5.41MB/s]
main/test-00000-of-00001.parquet: 100%                 419k/419k [00:00<00:00, 1.75MB/s]
Generating train split: 100%                          7473/7473 [00:00<00:00, 75055.93 examples/s]
Generating test split: 100%                            1319/1319 [00:00<00:00, 36756.21 examples/s]
```

LOAD MODEL

```
generator = pipeline(
    "text-generation",
    model="google/flan-t5-small"
)
print("Model loaded successfully!")
```

```

model.safetensors: 100% 308M/308M [00:02<00:00, 187MB/s]

Loading weights: 100% 190/190 [00:00<00:00, 489.33it/s, Materializing param=shared.weight]
The tied weights mapping and config for this model specifies to tie shared.weight to lm_head.weight, but both are present in the
generation_config.json: 100% 147/147 [00:00<00:00, 6.13kB/s]

tokenizer_config.json: 2.54k/? [00:00<00:00, 117kB/s]

spiece.model: 100% 792k/792k [00:00<00:00, 3.24MB/s]

tokenizer.json: 2.42M/? [00:00<00:00, 33.8MB/s]

special_tokens_map.json: 2.20k/? [00:00<00:00, 148kB/s]
The model 'T5ForConditionalGeneration' is not supported for text-generation. Supported models are ['PeftModelForCausalLM', 'Afmo
Model loaded successfully!

```

✓ PROMPT FUNCTIONS

```

# Zero-shot Prompt
def zero_shot_prompt(question):
    return f"""
Solve the following math problem.
Give only the final answer.

Question: {question}
"""

# Few-shot Prompt
def few_shot_prompt(question):
    return f"""
Solve the following math problems.

Example 1:
Question: A boy has 2 apples and buys 3 more. How many apples?
Answer: 5

Example 2:
Question: There are 4 boxes with 5 items each. Total items?
Answer: 20

Now solve:

Question: {question}
Answer:
"""

# Chain-of-Thought Prompt
def cot_prompt(question):
    return f"""
Solve the following math problem step by step.
Then give the final answer clearly.

Question: {question}
"""

```

✓ RESPONSE FUNCTION

```

def get_response(prompt):
    output = generator(
        prompt,
        max_length=128,
        do_sample=False
    )
    return output[0]["generated_text"]

```

✓ EXTRACT NUMBER FUNCTION

```
def extract_number(text):
    numbers = re.findall(r'[-]?[0-9]+\.[0-9]*', text.replace(",", ""))
    return numbers[-1] if numbers else None
```

- ✓ RUN EXPERIMENT

```

results = []

for i, sample in enumerate(samples):
    question = sample["question"]
    true_answer_text = sample["answer"]

    # Extract true answer
    true_answer = extract_number(true_answer_text)

    # Zero-shot
    z_res = get_response(zero_shot_prompt(question))
    z_pred = extract_number(z_res)

    results.append({
        "Q.No": i+1,
        "Prompt Type": "Zero-shot",
        "Question": question,
        "Model Response": z_res,
        "Actual Answer": true_answer,
        "Predicted Answer": z_pred
    })

    # Few-shot
    f_res = get_response(few_shot_prompt(question))
    f_pred = extract_number(f_res)

    results.append({
        "Q.No": i+1,
        "Prompt Type": "Few-shot",
        "Question": question,
        "Model Response": f_res,
        "Actual Answer": true_answer,
        "Predicted Answer": f_pred
    })

    # Chain-of-Thought
    c_res = get_response(cot_prompt(question))
    c_pred = extract_number(c_res)

    results.append({
        "Q.No": i+1,
        "Prompt Type": "Chain-of-Thought",
        "Question": question,
        "Model Response": c_res,
        "Actual Answer": true_answer,
        "Predicted Answer": c_pred
    })

print("Experiment completed!")

```

[illegible]

✓ CREATE DATAFRAME

```
df = pd.DataFrame(results)
```

✓ EVALUATION

```
df["Correct/Incorrect"] = df["Actual Answer"] == df["Predicted Answer"]
```

✓ FINAL EVALUATION TABLE

```
final_table = df[[
    "Q.No",
    "Prompt Type",
    "Actual Answer",
    "Predicted Answer",
    "Correct/Incorrect"
]]

print("\nFinal Evaluation Table:")
print(final_table)
```

```
Final Evaluation Table:
   Q.No  Prompt Type Actual Answer Predicted Answer Correct/Incorrect
0     1    Zero-shot           18              2           False
1     1    Few-shot           18              2           False
2     1 Chain-of-Thought          18              2           False
3     2    Zero-shot            3              2           False
4     2    Few-shot            3              2           False
5     2 Chain-of-Thought            3              2           False
6     3    Zero-shot          70000             150           False
7     3    Few-shot          70000             150           False
8     3 Chain-of-Thought          70000            1000           False
9     4    Zero-shot           540              60           False
10    4    Few-shot           540              3           False
11    4 Chain-of-Thought           540              3           False
12    5    Zero-shot            20             20            True
13    5    Few-shot            20             20            True
14    5 Chain-of-Thought            20             20            True
```

✓ ACCURACY CALCULATION

```
accuracy = df.groupby("Prompt Type")["Correct/Incorrect"].mean() * 100
accuracy = accuracy.reset_index()
accuracy.columns = ["Prompt Type", "Accuracy (%)"]

print("\nAccuracy Comparison:")
print(accuracy)
```

```
Accuracy Comparison:
   Prompt Type  Accuracy (%)
0 Chain-of-Thought         20.0
1    Few-shot         20.0
2    Zero-shot         20.0
```

✓ PLOT GRAPH

```
plt.figure(figsize=(8,5))
plt.bar(accuracy["Prompt Type"], accuracy["Accuracy (%)"])
plt.xlabel("Prompt Type")
plt.ylabel("Accuracy (%)")
plt.title("Accuracy Comparison of Prompting Techniques")
plt.show()
```

